Chapter 4 Homework

4.2 Implement Program 4.4 as a recursive-descent parser, with the semantic actions embedded in the parsing functions.

```
%{
typedef struct table *Table_;
Table_ {string id; int value; Table_ tail};
Table_ Table(string id, int value, struct table *tail); (see page 13)
Table_ table=NULL;
int lookup(Table_ table, string id) {
assert(table!=NULL);
if (id==table.id) return table.value;
else return lookup(table.tail, id);
}
void update(Table_ *tabptr, string id, int value) {
*tabptr = Table(id, value, *tabptr);
}
%}
%union {int num; string id;}
%token <num> INT
%token <id> ID
%token ASSIGN PRINT LPAREN RPAREN
%type <num> exp
%right SEMICOLON
%left PLUS MINUS
%left TIMES DIV
%start prog
%%
prog: stm
stm : stm SEMICOLON stm
stm : ID ASSIGN exp {update(&table,ID,$3);}
stm : PRINT LPAREN exps RPAREN {printf("\n");}
exps: exp {printf("%d ", $1);}
exps: exps COMMA exp {printf("%d ", $3);}
exp : INT {$$=$1;}
exp : ID {$$=lookup(table,$1);}
exp : exp PLUS exp {$$=$1+$3;}
exp : exp MINUS exp {$$=$1-$3;}
exp : exp TIMES exp {$$=$1*$3;}
exp : exp DIV exp {$$=$1/$3;}
exp : stm COMMA exp {$$=$3;}
exp : LPAREN exp RPAREN {$$=$2;}
```

解答:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

typedef struct Table {
        char *id;
        int value;
        struct Table *tail;
} Table;

Table *table = NULL;

void update(Table **tabptr, const char *id, int value) {
        Table *t = malloc(sizeof(*t));
        t->id = strdup(id);
        t->value = value;
        t->tail = *tabptr;
        *tabptr = t;
}

int lookup(Table *tab, const char *id) {
        for (; tab; tab = tab->tail) {
                if (strcmp(tab->id, id) == 0) return tab->value;
        }
        fprintf(stderr, "warning: undefined id '%s', returning 0\n", id);
        return 0;
}

/* Lexer */
typedef enum {TOK_EOF, TOK_INT, TOK_ID, TOK_ASSIGN, TOK_PRINT,
                        TOK_LPAREN, TOK_RPAREN, TOK_COMMA, TOK_SEMI,
                        TOK_PLUS, TOK_MINUS, TOK_TIMES, TOK_DIV} Token;

char lex_id[128];
int lex_num;
int c = ' ';

void nextch(){ c = getchar(); }

void skipws() { while (isspace(c)) nextch(); }

int is_ident_start(int ch){ return isalpha(ch) || ch=='_'; }
```

```
int is_ident_body(int ch){ return isalnum(ch) || ch=='_'; }

Token lookahead;

Token nextToken() {
        skipws();
        if (c==EOF || c== -1) return TOK_EOF;
        if (isdigit(c)) {
                int v = 0;
                while (isdigit(c)) { v = v*10 + (c-'0'); nextch(); }
                lex_num = v;
                return TOK_INT;
        }
        if (is_ident_start(c)) {
                int i=0;
                while (is_ident_body(c) && i < (int)sizeof(lex_id)-1) {
                        lex_id[i++] = c; nextch();
                }
                lex_id[i]=0;
                if (strcmp(lex_id,"print")==0) return TOK_PRINT;
                return TOK_ID;
        }
        int ch = c;
        nextch();
        switch (ch) {
                case '=': return TOK_ASSIGN;
                case '(': return TOK_LPAREN;
                case ')': return TOK_RPAREN;
                case ',': return TOK_COMMA;
                case ';': return TOK_SEMI;
                case '+': return TOK_PLUS;
                case '-': return TOK_MINUS;
                case '*': return TOK_TIMES;
                case '/': return TOK_DIV;
        }
        return TOK_EOF;
}

void consume(Token t) {
        if (lookahead == t) lookahead = nextToken();
        else { fprintf(stderr,"syntax error: unexpected token\n"); exit(1); }
}
```

```
int parse_exp();

int parse_factor() {
        if (lookahead == TOK_INT) {
                int v = lex_num;
                consume(TOK_INT);
                return v;
        } else if (lookahead == TOK_ID) {
                char idbuf[128];
                strcpy(idbuf, lex_id);
                consume(TOK_ID);
                return lookup(table, idbuf);
        } else if (lookahead == TOK_LPAREN) {
                consume(TOK_LPAREN);
                int v = parse_exp();
                consume(TOK_RPAREN);
                return v;
        } else {
                fprintf(stderr,"syntax error in factor\n"); exit(1);
        }
}

int parse_term() {
        int v = parse_factor();
        while (lookahead == TOK_TIMES || lookahead == TOK_DIV) {
                if (lookahead == TOK_TIMES) { consume(TOK_TIMES); v *= parse_factor(
                else { consume(TOK_DIV); v /= parse_factor(); }
        }
        return v;
}

int parse_exp() {
        int v = parse_term();
        while (lookahead == TOK_PLUS || lookahead == TOK_MINUS) {
                if (lookahead == TOK_PLUS) { consume(TOK_PLUS); v += parse_term(); }
                else { consume(TOK_MINUS); v -= parse_term(); }
        }
        return v;
}

void parse_exps_and_print() {
        int v = parse_exp();
        printf("%d ", v);
```

```
        while (lookahead == TOK_COMMA) {
                consume(TOK_COMMA);
                v = parse_exp();
                printf("%d ", v);
        }
}

void parse_stmt() {
        if (lookahead == TOK_ID) {
                char idbuf[128];
                strcpy(idbuf, lex_id);
                consume(TOK_ID);
                consume(TOK_ASSIGN);
                int v = parse_exp();
                update(&table, idbuf, v);
        } else if (lookahead == TOK_PRINT) {
                consume(TOK_PRINT);
                consume(TOK_LPAREN);
                if (lookahead != TOK_RPAREN) {
                        parse_exps_and_print();
                }
                consume(TOK_RPAREN);
                printf("\n");
        } else {
                fprintf(stderr,"syntax error: unknown statement\n"); exit(1);
        }
}

void parse_prog() {
        lookahead = nextToken();
        if (lookahead == TOK_EOF) return;
        parse_stmt();
        while (lookahead == TOK_SEMI) {
                consume(TOK_SEMI);
                if (lookahead == TOK_EOF) break;
                parse_stmt();
        }
        if (lookahead != TOK_EOF) {
                fprintf(stderr,"syntax error: expected EOF\n");
                exit(1);
        }
}
```

```
int main() {
          nextch();
          parse_prog();
          return 0;
}
```

Chapter 5 Homework

5.1 Improve the hash table implementation of Program 5.2:

a. Double the size of the arraywhen the average bucket length grows larger than 2 (so table is now a pointer to a dynamically allocated array). To double an array, allocate a bigger one and rehash the contents of the old array; then discard the old array.
b. Allow for more than one table to be in use bymaking the table a parameter to insert and lookup.

```
struct bucket {string key; void *binding; struct bucket *next;};
#define SIZE 109
struct bucket *table[SIZE];
unsigned int hash(char *s0)
{unsigned int h=0; char *s;
for(s=s0; *s; s++)
h = h*65599 + *s;
return h;
}
struct bucket *Bucket(string key, void *binding, struct bucket *next) {
struct bucket *b = checked_malloc(sizeof(*b));
b->key=key; b->binding=binding; b->next=next;
return b;
}
void insert(string key, void *binding) {
int index = hash(key) % SIZE;
table[index] = Bucket(key, binding, table[index]);
}
void *lookup(string key) {
int index = hash(key) % SIZE;
struct bucket *b;
for(b=table[index]; b; b=b->next)
if (0==strcmp(b->key,key)) return b->binding;
return NULL;
}
void pop(string key) {
int index = hash(key) % SIZE;
table[index] = table[index]->next;
}
```

解答:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct bucket {
	char *key;
	void *binding;
	struct bucket *next;
} bucket;

typedef struct {
	int size;
	int n;
	bucket **buckets;
} HashTable;

unsigned int hash_str(const char *s) {
	unsigned int h = 0;
	for (const unsigned char *p = (const unsigned char*)s; *p; p++)
		h = h * 65599u + *p;
	return h;
}

HashTable *ht_create(int init_size) {
	HashTable *ht = malloc(sizeof(*ht));
	ht->size = init_size;
	ht->n = 0;
	ht->buckets = calloc(ht->size, sizeof(bucket*));
	return ht;
}

bucket *bucket_new(const char *key, void *binding, bucket *next) {
	bucket *b = malloc(sizeof(*b));
	b->key = strdup(key);
	b->binding = binding;
	b->next = next;
	return b;
}

void ht_rehash(HashTable *ht, int new_size) {
	bucket **old = ht->buckets;
	int old_size = ht->size;
	ht->buckets = calloc(new_size, sizeof(bucket*));
```

```
        ht->size = new_size;
        for (int i = 0; i < old_size; ++i) {
                bucket *b = old[i];
                while (b) {
                        bucket *next = b->next;
                        unsigned int idx = hash_str(b->key) % ht->size;
                        b->next = ht->buckets[idx];
                        ht->buckets[idx] = b;
                        b = next;
                }
        }
        free(old);
}

void ht_insert(HashTable *ht, const char *key, void *binding) {
        unsigned int idx = hash_str(key) % ht->size;
        ht->buckets[idx] = bucket_new(key, binding, ht->buckets[idx]);
        ht->n++;
        double avg = (double)ht->n / (double)ht->size;
        if (avg > 2.0) {
                ht_rehash(ht, ht->size * 2);
        }
}

void *ht_lookup(HashTable *ht, const char *key) {
        unsigned int idx = hash_str(key) % ht->size;
        for (bucket *b = ht->buckets[idx]; b; b = b->next) {
                if (strcmp(b->key, key) == 0) return b->binding;
        }
        return NULL;
}

int ht_remove(HashTable *ht, const char *key) {
        unsigned int idx = hash_str(key) % ht->size;
        bucket *b = ht->buckets[idx];
        bucket *prev = NULL;
        while (b) {
                if (strcmp(b->key, key) == 0) {
                        if (prev) prev->next = b->next;
                        else ht->buckets[idx] = b->next;
                        free(b->key);
                        free(b);
                        ht->n--;
```

```
                        return 1;
                }
                prev = b;
                b = b->next;
        }
        return 0;
}

int main() {
        HashTable *ht = ht_create(8);
        for (int i = 0; i < 30; ++i) {
                char buf[32];
                sprintf(buf, "k%d", i);
                int *v = malloc(sizeof(int));
                *v = i * 10;
                ht_insert(ht, buf, v);
        }
        printf("Inserted %d items into table with size %d, load avg %.2f\n",
                        ht->n, ht->size, (double)ht->n / ht->size);
        int *pv = ht_lookup(ht, "k17");
        if (pv) printf("k17 -> %d\n", *pv);
        ht_remove(ht, "k17");
        if (!ht_lookup(ht, "k17")) printf("k17 removed\n");
        return 0;
}
```